

Machine Learning

Dr. Irfan Yousuf

Department of Computer Science (New Campus)

UET, Lahore

(Week 10; November 17 – 21, 2025)

Outline

- What is Convolution?
- Convolutional Neural Networks

Convolution

- In mathematics (in particular, functional analysis), convolution is a mathematical operation on two functions f and g that produces a third function $f*g$. The term convolution refers to both the result function and to the process of computing it.
- It is defined as the **integral** of the **product** of two functions after one is **reflected** about the y-axis and **shifted**.

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t - \tau)d\tau$$

Convolution: Hospital Analogy

- Convolution is fancy multiplication.
- Imagine you manage a hospital treating patients with a single disease. You have:
 - A treatment plan: `[3]` Every patient gets 3 units of the cure on their first day.
 - A list of patients: `[1 2 3 4 5]` Your patient count for the week (1 person Monday, 2 people on Tuesday, etc.).

Convolution: Hospital Analogy

- Question: How much medicine do you use each day?

```
Plan * Patients = Daily Usage  
[3] * [1 2 3 4 5] = [3 6 9 12 15]
```

Convolution: Hospital Analogy

- Let's say the disease mutates and requires a multi-day treatment. You create a new plan:

Plan: [3 2 1]

- That means 3 units of the cure on the first day, 2 on the second, and 1 on the third.
- Given the same patient schedule [1 2 3 4 5], what's our medicine usage each day

Convolution: Hospital Analogy

- On Monday, 1 patient comes in. It's her first day, so she gets 3 units.
- On Tuesday, the Monday gal gets 2 units (her second day), but two new patients arrive, who get 3 each ($2 * 3 = 6$). The total is $2 + (2 * 3) = 8$ units.
- On Wednesday, it's trickier: The Monday gal finishes (1 unit, her last day), the Tuesday people get 2 units ($2 * 2$), and there are 3 new Wednesday people... argh.

The patients are overlapping and it's hard to track. How can we organize this calculation?

Convolution: Hospital Analogy

An idea: imagine *flipping* the patient list, so the first patient is on the right:

```
Start of line
5 4 3 2 1
```

Next, imagine we have 3 separate rooms where we apply the proper dose:

```
Rooms      3 2 1
```

On first day, a patient walks into the first room and gets 3 units of medicine. The next day, he walks into room #2 and gets 2 units. On the last day, he walks into room #3 and gets 1 unit. There's no rooms afterwards, and treatment is done.

Convolution: Hospital Analogy

To calculate the total medicine usage, line up the patients and walk them through the rooms:

Monday

.....

Rooms	3	2	1
-------	---	---	---

Patients	5	4	3	2	1
----------	---	---	---	---	---

Usage	3
-------	---

Tuesday

.....

Rooms	3	2	1
-------	---	---	---

Patients -> 5 4 3 2 1

Usage 6 2 = 8

Convolution: Hospital Analogy

To calculate the total medicine usage, line up the patients and walk them through the rooms:

Wednesday

Rooms 3 2 1

Patients -> 5 4 3 2 1

Usage 9 4 1 = 14

Thursday

Rooms 3 2 1

Patients -> 5 4 3 2 1

Usage 12 6 2 = 20

Friday

Rooms 3 2 1

Patients -> 5 4 3 2 1

Usage 15 8 3 = 26

Convolution: Hospital Analogy

$$\begin{array}{rcl} \text{Plan} & * & \text{Patient List} & = & \text{Total Daily Usage} \\ \\ [3 \ 2 \ 1] & * & [1 \ 2 \ 3 \ 4 \ 5] & = & [3 \ 8 \ 14 \ 20 \ 26 \ 14 \ 5] \\ & & \text{M T W T F} & & \text{M T W T F S S} \end{array}$$

This calculation is the *convolution* of the plan and patient list. It's a fancy multiplication between a list of input numbers and a "program".

Convolution

- Convolution reverses one of the lists! Here's why.

Let's call our treatment plan $f(x)$. In our example, we used `[3 2 1]`.

The list of patients (inputs) is $g(x)$. However, we need to *reverse* this list as we slide it, so the earliest patient (Monday) enters the hospital first (first in, first out). This means we need to use $g(-x)$, the horizontal reflection of $g(x)$. `[1 2 3 4 5]` becomes `[5 4 3 2 1]`.

Convolution

Now that we have the reversed list, pick a day to compute ($t = 0, 1, 2 \dots$). To slide our patient list by this much, we use: $g(-x + t)$. That is, we reverse the list ($-x$) and jump to the correct day ($+t$).

We have our scenario:

- $f(x)$ is the plan to use
- $g(-x + t)$ is the list of inputs (flipped and slid to the right day).

To get the total usage on day t , we multiply each patient with the plan, and sum the results (an integral). To account for any possible length, we go from -infinity to +infinity.

Convolution

$$(f * g)(t) \stackrel{\text{def}}{=} \int_{-\infty}^{\infty} f(\tau)g(t - \tau) d\tau$$

To convolve a kernel with an input signal:
flip the signal, move to the desired time,
and accumulate every interaction with the kernel

Convolution is commutative: $f * g = g * f$

In our computation, we flipped the patient list and kept the plan the same. Could we have flipped the plan instead?

You bet. Imagine the patients are immobile, and stay in their rooms: `[1 2 3 4 5]`. To deliver the medicine, we have 3 medical carts that go to each room and deliver the dose. Each day, they slide forward one position.

Carts ->

1 2 3

1 2 3 4 5

Patients

Convolution: Kernel

- The trick with convolution is finding a useful "program" (kernel) to apply to your input.
- Let's say you want a moving average between neighboring items in a list. That is half of each element, added together:

list = {1 4 9 16 25}

Convolution: Kernel

- Multiplication Program (Kernel) of $[0.5 \quad 0.5]$ convolved with our list:

list = {1 4 9 16 25}

Convolution: Kernel

- The derivative finds the difference between neighboring values. Here's the kernel: $[1 \ -1]$

$\text{list} = \{1 \ 4 \ 9 \ 16 \ 25\}$

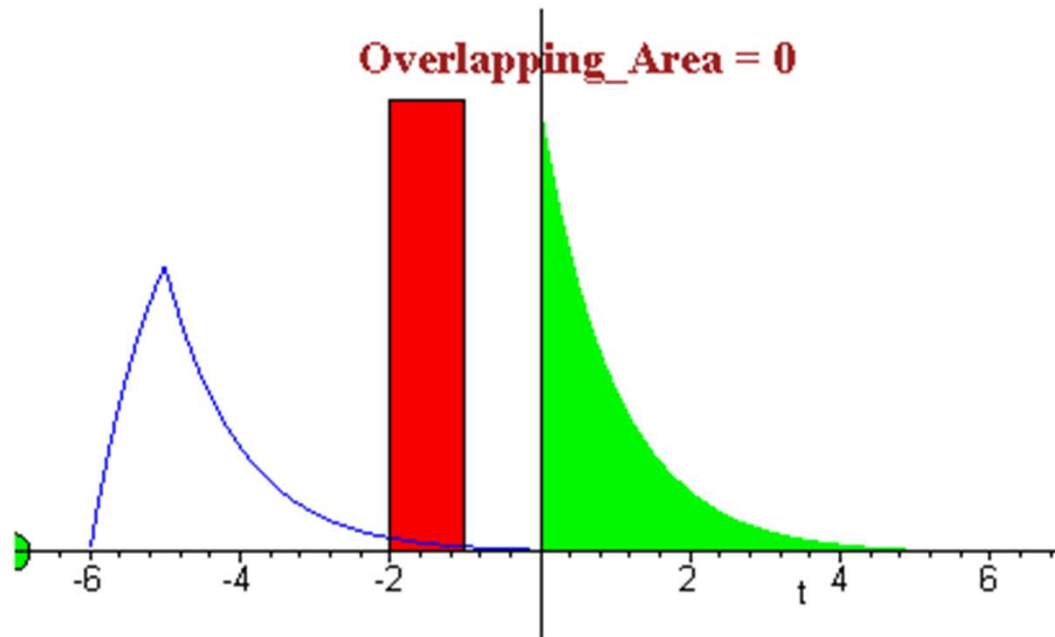
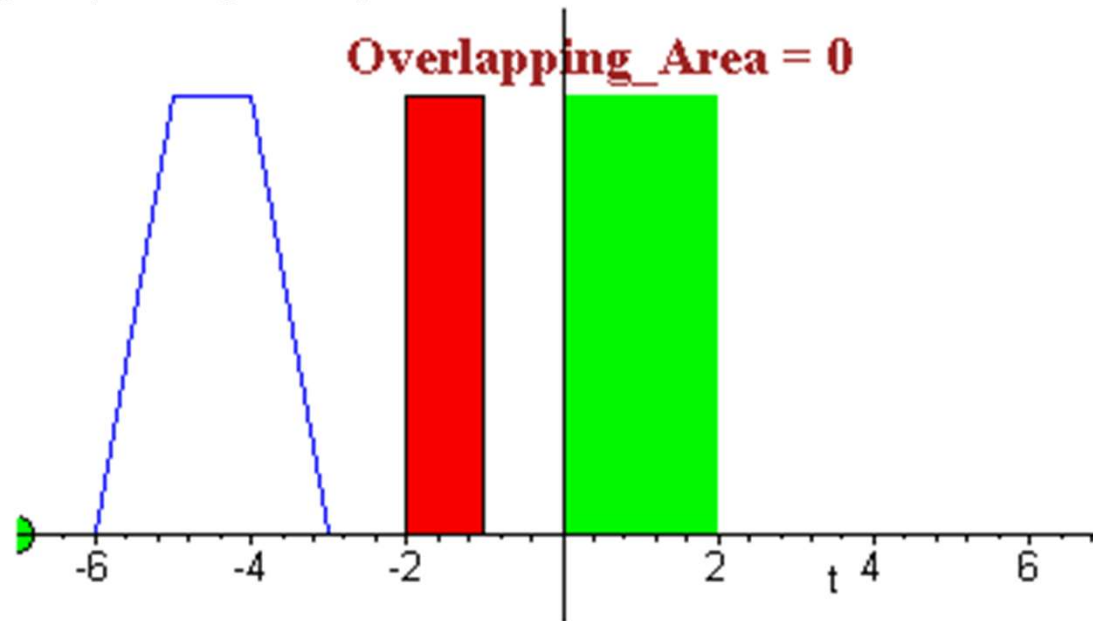
Convolution: Kernel

An image blur is essentially a convolution of your image with some "blurring kernel":

$$\text{blurred} = \text{image} * \text{blur}$$

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

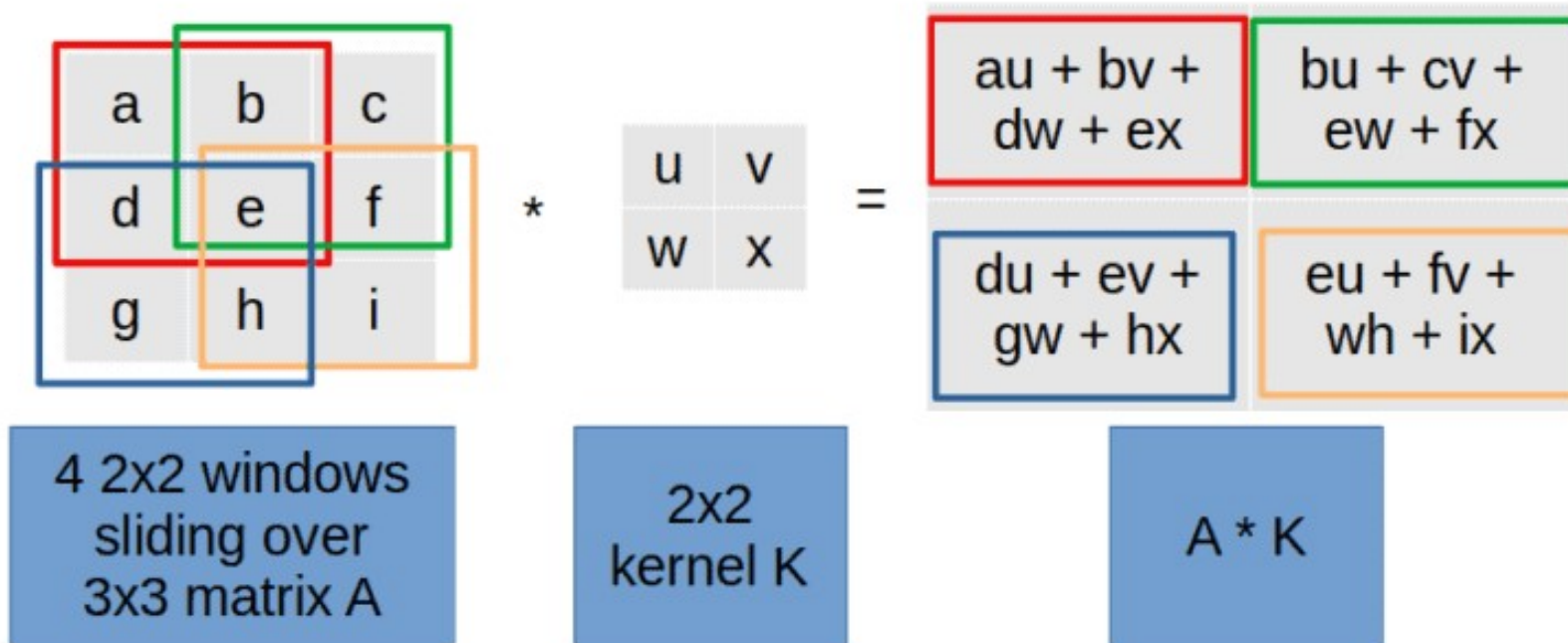
Convolution: Kernel



Convolution in 2D

- Convolution is fundamental in signal processing, computer vision, and machine learning. It applies a **filter or kernel** to an input image or signal and extracts relevant features.
- Typical implementations use a sliding-window operation where the kernel moves across the input image. For each placement of the kernel K on the input image A , we compute the dot product of the kernel and the overlapping image pixels:

Convolution in 2D

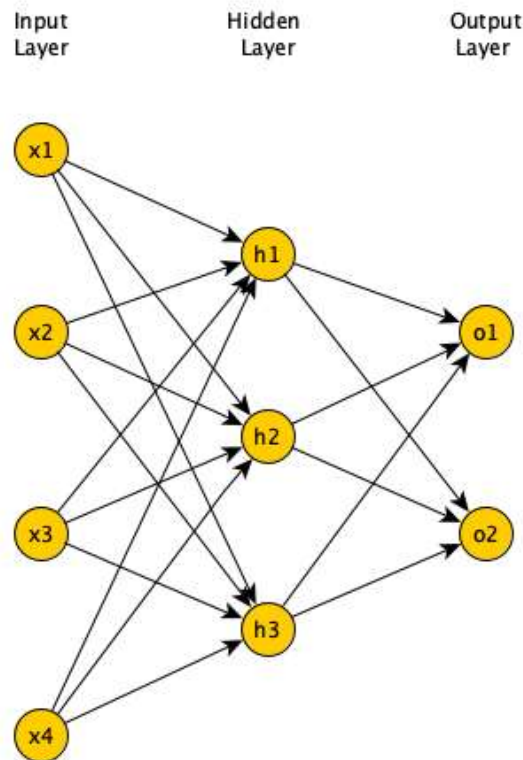


Convolutional Neural Networks

- Increasing popularity of social media in past decade
 - Image and video processing tasks have become very important
- Feedforward Neural Networks (FNN) could not scale up to image and video processing tasks.
- CNN specifically tailored for image and video processing tasks

Feedforward Neural Networks

- In FNN all nodes in a layer connected to all nodes in next layer.
- Each connection has a weight, must be learned by learning algorithm.



Limitations of Feedforward Neural Networks

- If input is 64 pixel by 64 pixel grayscale image:
 - Each grayscale pixel represented by 1 value, usually between 0 to 255
 - Where 0 is black, 255 is white, and values in between are shades of gray
 - Since each grayscale pixel represented by 1 value, we say channel size is 1
- Image represented by $64 \times 64 \times 1 = 4,096$ values (rows x columns x channels)
- Hence, input layer of FNN has 4096 nodes
- Let's assume next layer has 500 nodes
- Since FNN fully connected, we have $4,096 \times 500 = 2,048,000$ weights

Limitations of Feedforward Neural Networks

- Clear that FNN cannot scale to larger images (Too many weights)
- Another problem with FNN
 - 2D image represented as 1D vector in input layer
 - Any **spatial** relationship in the data is ignored

Inspiration for CNN

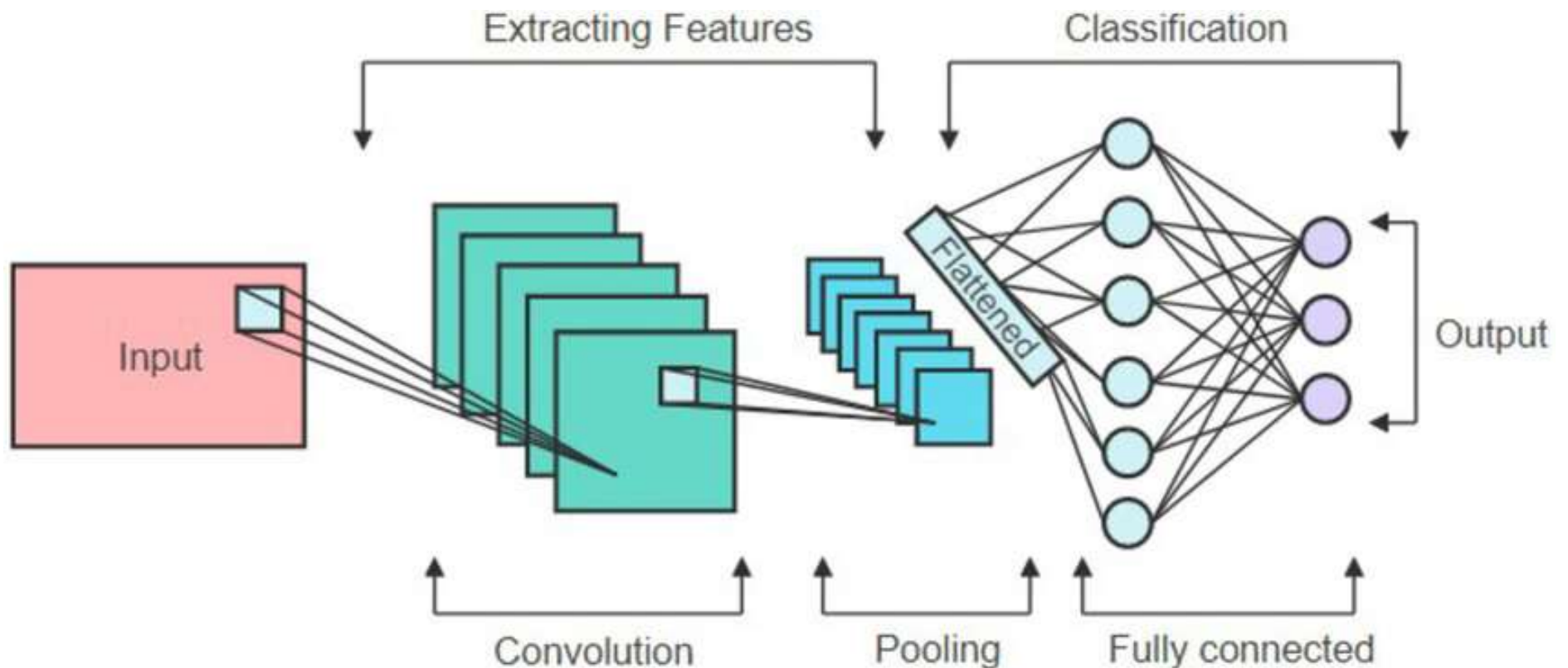
- In 1959 Hubel & Wiesel did an experiment to understand how visual cortex of brain processes visual info
- Recorded activity of neurons in visual cortex of a cat
- While moving a bright line in front of the cat
 - Some cells fired when bright line is shown at a particular angle/location
 - Called these simple cells
- Other cells fired when bright line was shown regardless of angle/location
 - Seemed to detect movement
 - Called these complex cells
- Seemed complex cells receive inputs from multiple simple cells and have an hierarchical structure
- Hubel and Wiesel won Noble prize in 1981

Inspiration for CNN

- Inspired by complex/simple cells, Fukushima proposed Neocognitron (1980)
 - Hierarchical neural network used for handwritten Japanese character recognition , own training algorithm
- In 1989, LeCun proposed CNN that was trained by backpropagation
- CNN got popular when outperformed other models at ImageNet Challenge
 - Competition in object classification/detection
 - On hundreds of object categories and millions of images
 - Run annually from 2010 to present
- Notable CNN architectures that won ImageNet challenge
 - AlexNet (2012), ZFNet (2013), GoogLeNet & VGG (2014), ResNet (2015)

Architecture of CNN

- A typical CNN has 4 layers
 - Input layer
 - Convolution layer
 - Pooling layer
 - Fully connected layer



Architecture of CNN: Input Layer

- Example input a 28 pixel by 28 pixel grayscale image.
- Unlike FNN, we do not “flatten” the input to a 1D vector.
- Input is presented to network in 2D as 28 x 28 matrix.
- This makes capturing spatial relationships easier.

Architecture of CNN: Convolution Layer

- Composed of multiple filters (kernels)
- Filters for 2D image are also 2D
- Suppose we have a 3 by 3 filter (9 values in total)
 - Values are randomly set to 0 or 1
- Convolution: placing 3 by 3 filter on the top left corner of image
 - Multiply filter values by pixel values, add the results
 - Move filter to right one pixel at a time, and repeat this process
 - When at top right corner, move filter down one pixel and repeat process
 - Process ends when we get to bottom right corner of image

Architecture of CNN: Convolution Layer

- Convolution operator parameters
 - Filter size
 - Padding
 - Stride
 - Dilation
 - Activation function

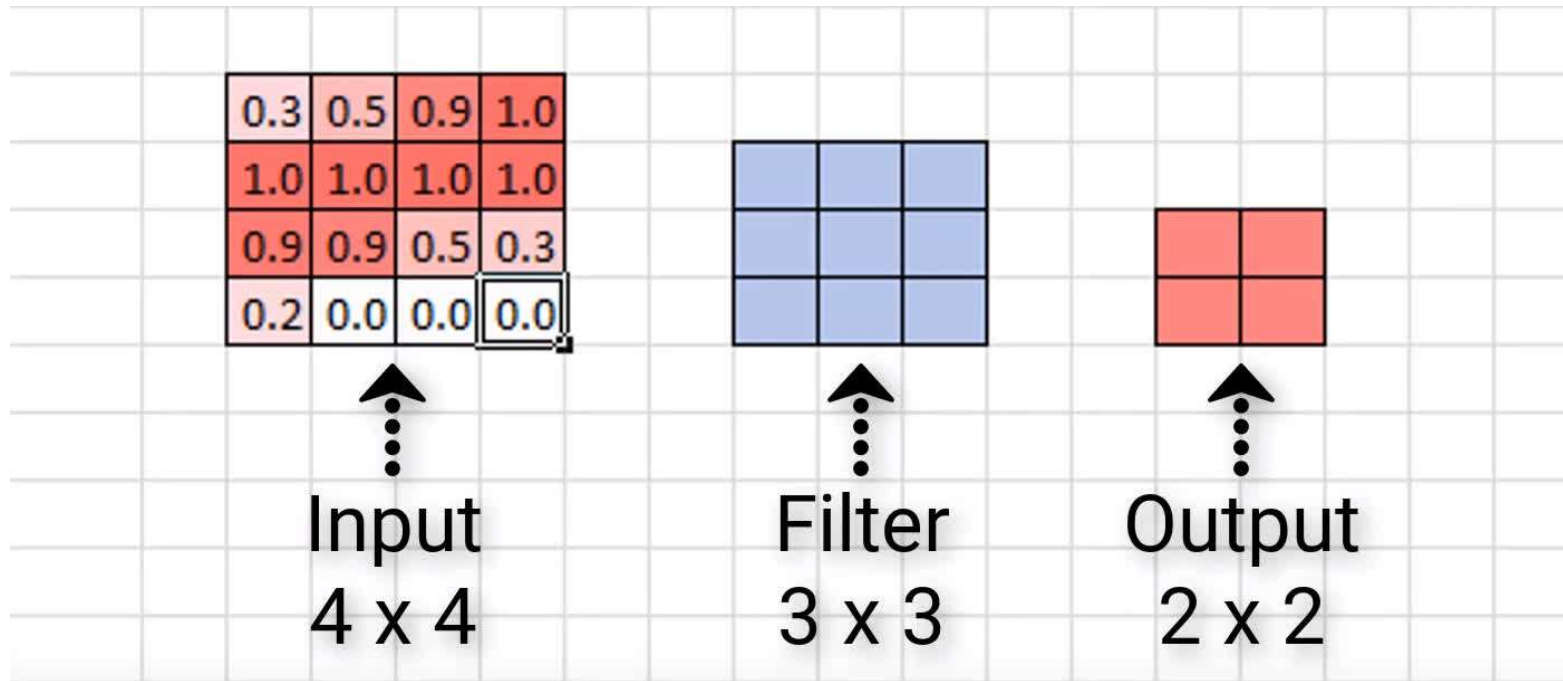
Convolution Layer: Filter Size

- Filter size can be 5 by 5, 3 by 3, and so on
- Larger filter sizes should be avoided
 - As learning algorithm needs to learn filter values (weights)
- Odd sized filters are preferred to even sized filters
 - Nice geometric property of all input pixels being around output pixel

Convolution Layer: Padding

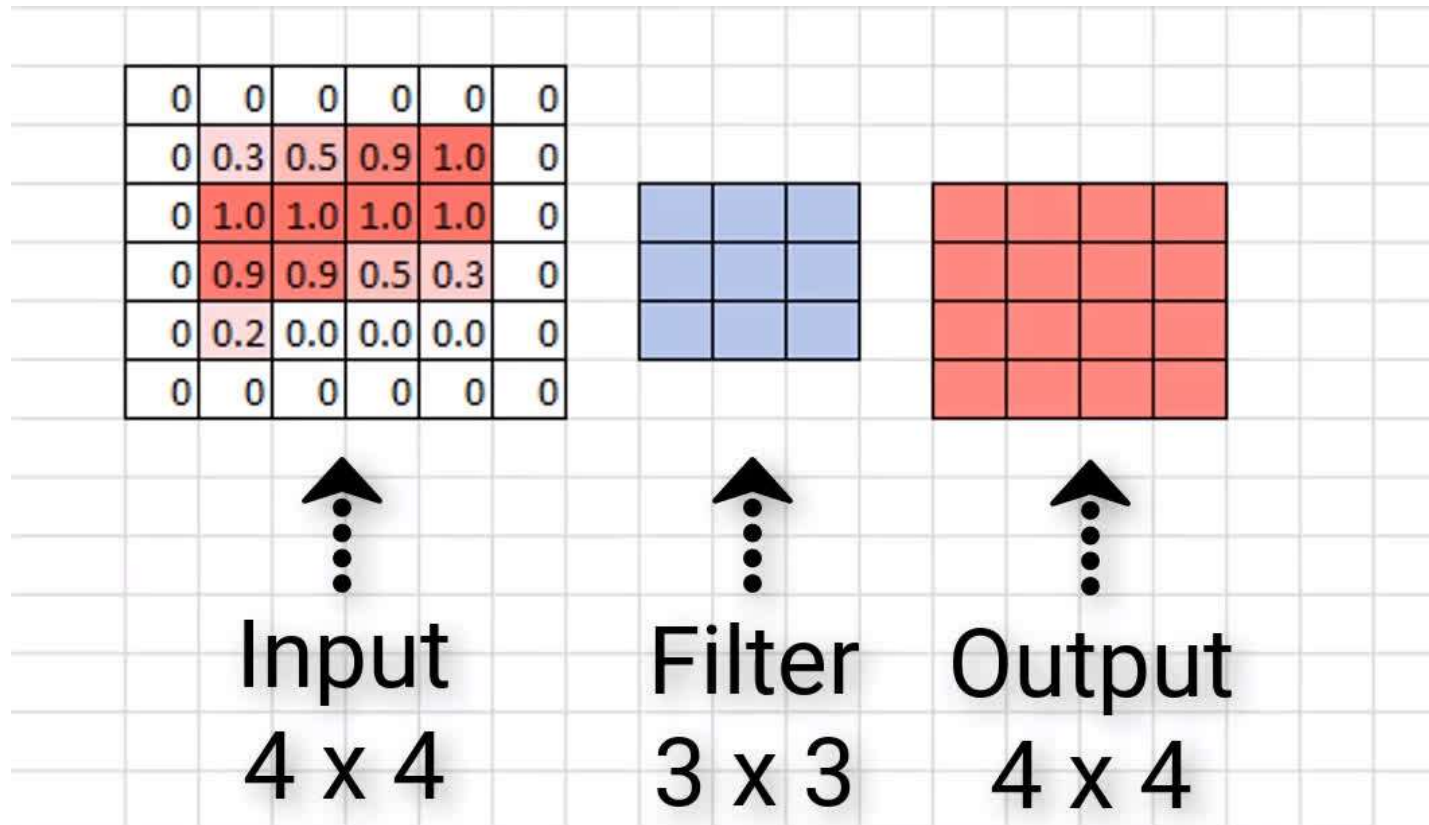
- After applying 3 by 3 filter to 4 by 4 image, we get a 2 by 2 image – Size of the image has gone down
- If we want to keep image size the same, we can use padding
 - We pad input in every direction with 0's before applying filter
 - If padding is 1 by 1, then we add 1 zero in every direction
 - If padding is 2 by 2, then we add 2 zeros in every direction, and so on

Convolution Layer: Padding



In general, if our image is of size $n \times n$, and we convolve it with an $f \times f$ filter, then the size of the resulting output is $(n - f + 1) \times (n - f + 1)$.

Convolution Layer: Padding



- Zero padding occurs when we add a border of pixels all with value zero around the edges of the input images. This adds kind of a padding of zeros around the outside of the image, hence the name zero padding.

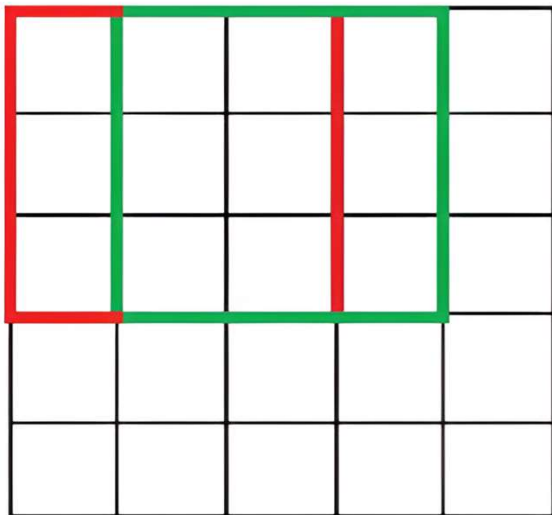
Convolution Layer: Padding

Padding Type	Description	Impact
Valid	No padding	Dimensions reduce
Same	Zeros around the edges	Dimensions stay the same

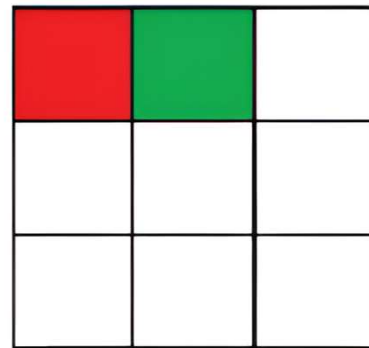
Convolution Layer: Stride

- How many pixels we move a filter to the right/down is called stride.
- Stride 1: move filter one pixel to the right/down
- Stride 2: move filter two pixels to the right/down

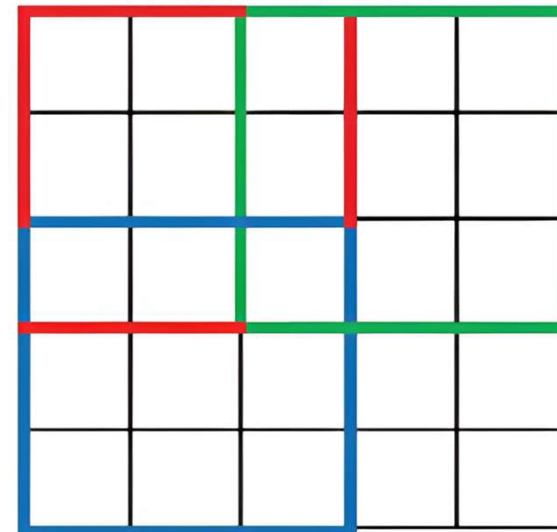
Convolution
with Stride=1



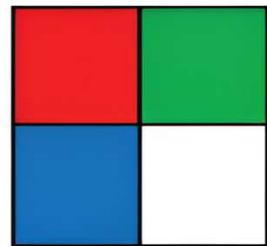
Output



Convolution
with Stride=2



Output



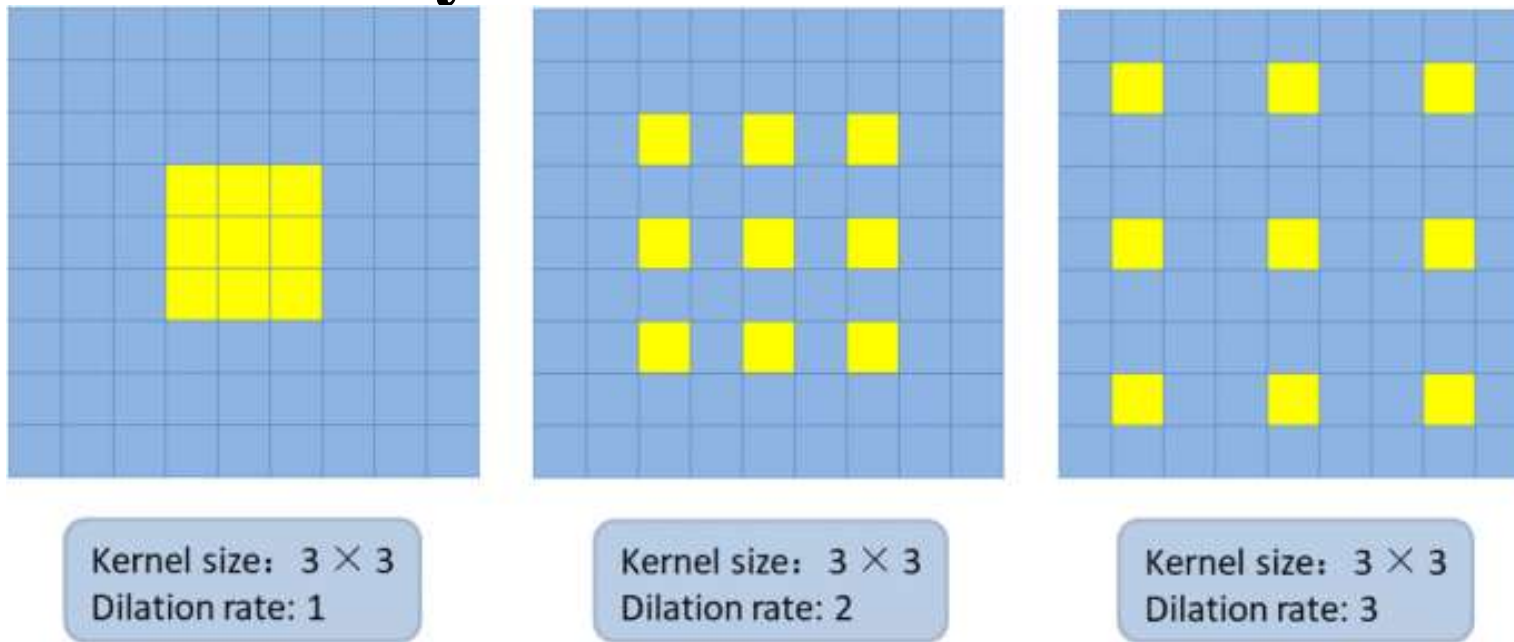
Convolution Layer: Dilation

- When we apply 3 by 3 filter, output affected by pixels in 3 by 3 subset of image.
- **Dilation:** To have a larger receptive field (portion of image affecting filter's output).
- If dilation is set to 2, instead of contiguous 3 by 3 subset of image, every other pixel of a 5 by 5 subset of image affects output

Convolution Layer: Dilation

- It is a technique that expands the kernel (input) by inserting holes between its consecutive elements. In simpler terms, it is the same as convolution but it involves pixel skipping, so as to cover a larger area of the input.
- In a dilated convolution operation, the filter is “dilated” by inserting **gaps** between the filter values. The dilation rate determines the size of the gaps, and it is a hyperparameter that can be adjusted. When the dilation rate is 1, the dilated convolution reduces to a regular convolution.

Convolution Layer: Dilation



The dilation rate effectively increases the receptive field of the filter without increasing the number of parameters, because the filter is still the same size, but with gaps between the values. This can be useful in situations where a larger receptive field is needed, but increasing the size of the filter would lead to an increase in the number of parameters and computational complexity.

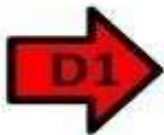
Convolution Layer: Dilation

Dilated
Convolution

Feature map

1	1	1	1	1	1	1
2	2	2	2	2	2	2
3	3	3	3	3	3	3
4	4	4	4	4	4	4
5	5	5	5	5	5	5
6	6	6	6	6	6	6
7	7	7	7	7	7	7

1	1	1
1	1	1
1	1	1



1	1	1	1	1	1	1
2	2	2	2	2	2	2
3	3	3	3	3	3	3
4	4	4	36	4	4	4
5	5	5	5	5	5	5
6	6	6	6	6	6	6
7	7	7	7	7	7	7

1		1		1
1		1		1
1		1		1



1	1	1	1	1	1	1
2	2	2	2	2	2	2
3	3	3	3	3	3	3
4	4	4	36	4	4	4
5	5	5	5	5	5	5
6	6	6	6	6	6	6
7	7	7	7	7	7	7

1			1			1
1			1			1
1			1			1



1	1	1	1	1	1	1
2	2	2	2	2	2	2
3	3	3	3	3	3	3
4	4	4	36	4	4	4
5	5	5	5	5	5	5
6	6	6	6	6	6	6
7	7	7	7	7	7	7

Convolution Layer: Activation Function

- After filter applied to whole image, apply activation function to output to introduce non-linearity.
- Preferred activation function in CNN is ReLU.
- ReLU leaves outputs with positive values as is, replaces negative values with 0

0	1	-4
0	2	0
-1	4	3

Filter output

→

0	1	0
0	2	0
0	4	3

Filter output after ReLU

Architecture of CNN: Pooling Layer

- Pooling layer performs down sampling to reduce spatial dimensionality of input
- This decreases number of parameters
 - Reduces learning time/computation
 - Reduces likelihood of overfitting
- Most popular type is max pooling
 - Usually a **2 x 2 filter with a stride of 2**
 - Returns maximum value as it slides over input data

Architecture of CNN: Pooling Layer

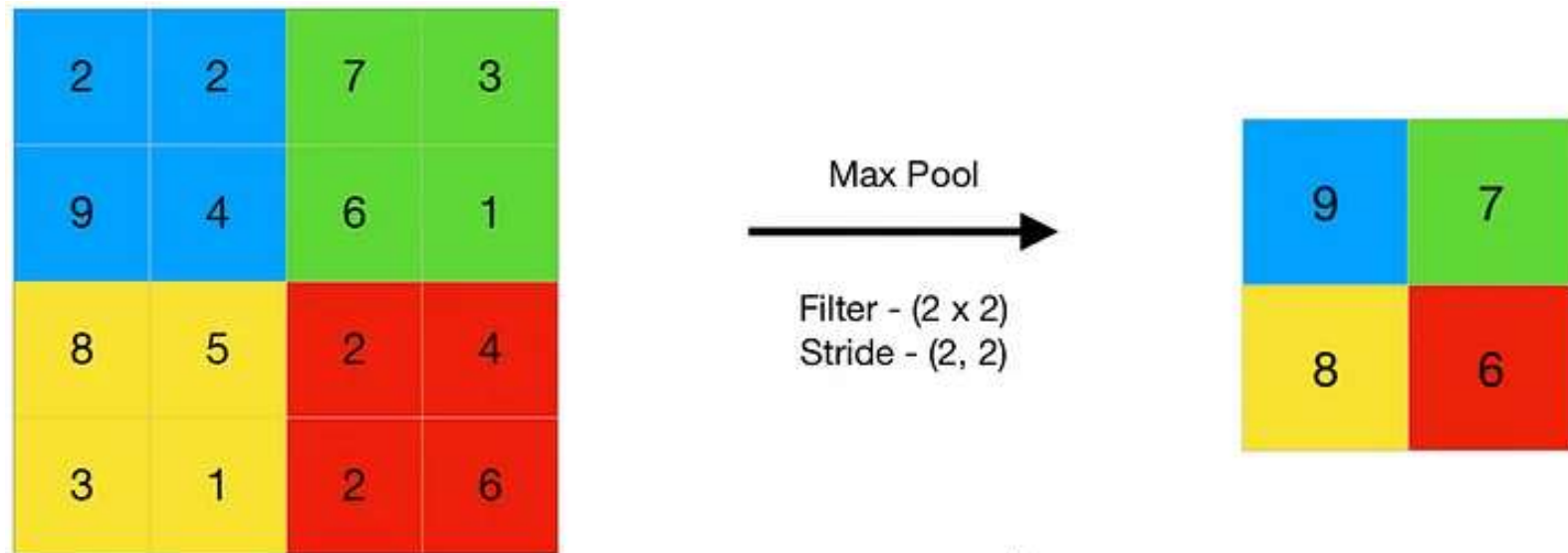
- Pooling, also known as subsampling or downsampling, is a technique used in CNNs to reduce the spatial dimensions of feature maps while retaining essential information.
- It helps in controlling the model's complexity, reducing overfitting, and improving computational efficiency by reducing the number of parameters and computation required in subsequent layers.

Architecture of CNN: Pooling Layer

- For a feature map having dimensions $h \times w \times c$, the dimensions of output obtained after a pooling layer is
- $(h-f+1) / s \times (w-f+1)/s \times c$
- h = height of feature map
- w = width of feature map
- c = number of channels in the feature map
- f = size of filter
- s = stride length

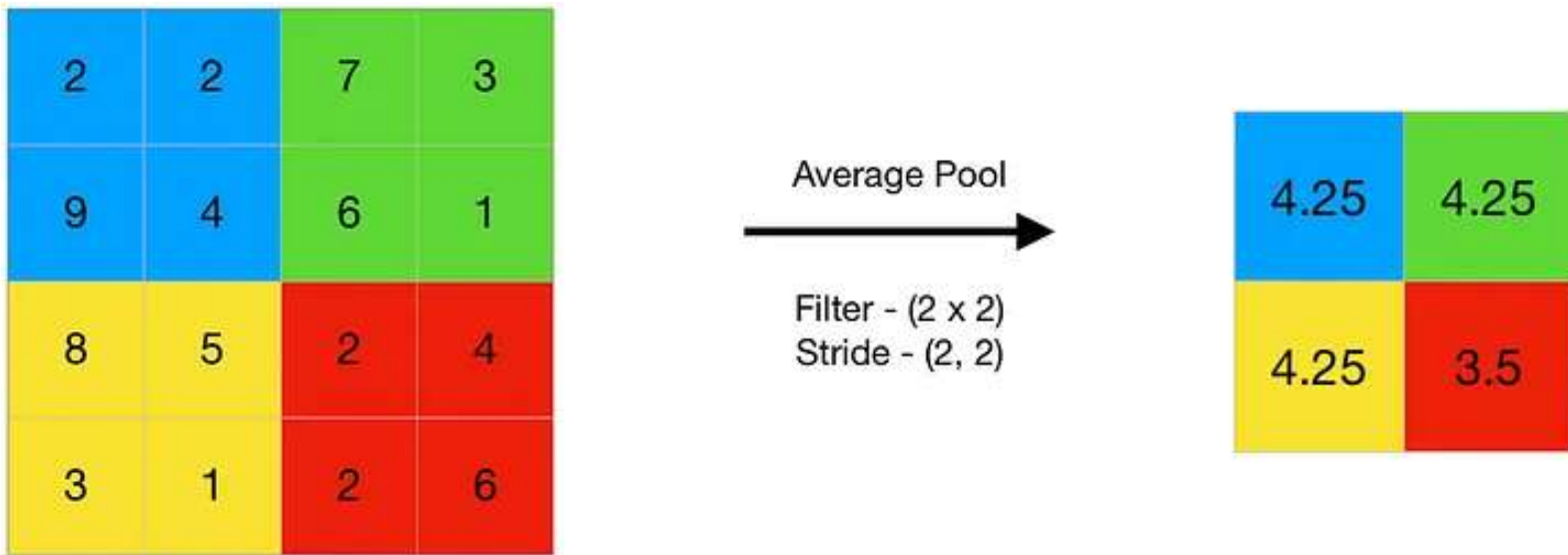
Architecture of CNN: Pooling Layer

- **Max Pooling:** Max pooling is a pooling operation that selects the maximum element from the region of the feature map covered by the filter.



Architecture of CNN: Pooling Layer

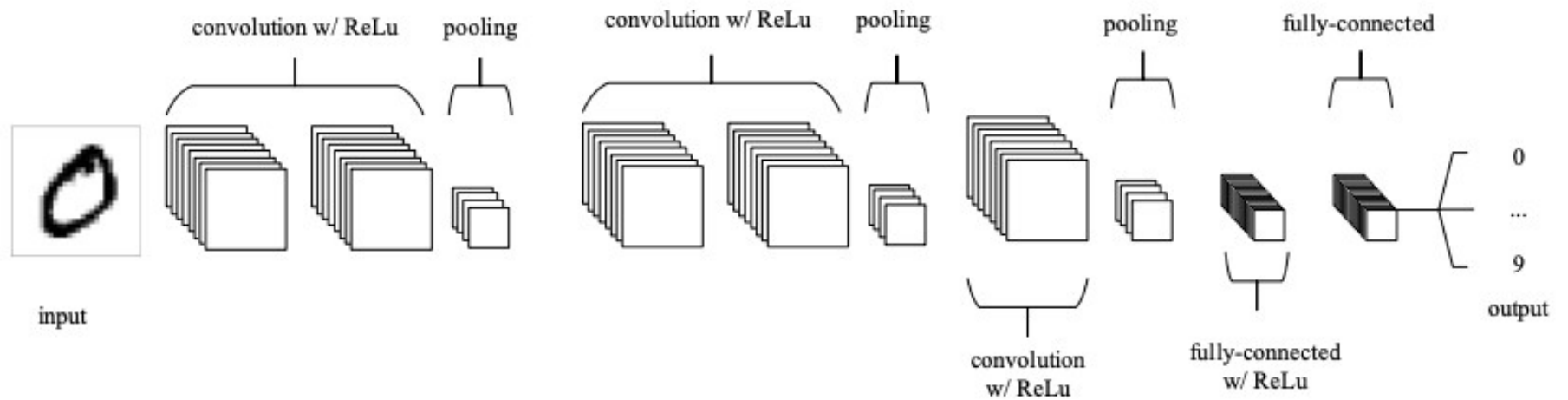
- **Average pooling** computes the average of the elements present in the region of feature map covered by the filter.



Architecture of CNN: Fully Connected Layer

- Last layer in a CNN
- Connect all nodes from previous layer to this fully connected layer
 - Which is responsible for classification of the image

Architecture of CNN



Architecture of CNN

- A typical CNN has several convolution plus pooling layers
- Each responsible for feature extraction at different levels of abstraction
 - E.g., filters in first layer detect horizontal, vertical, and diagonal edges
 - Filters in the next layer detect shapes
 - Filters in the last layer detect collection of shapes
- Filter values randomly initialized, learned by learning algorithm
- CNN not only do classification, but can also automatically do feature extraction
 - Distinguishes CNN from other classification techniques (like Support Vector Machines)

Summary

- What is Convolution
- Convolution Neural Networks